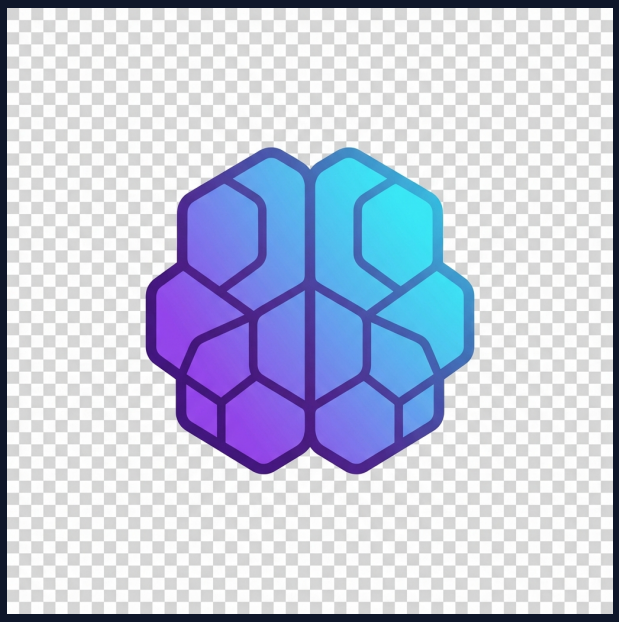


NeuralAI

Custom LLM with RAG + Terminal + Neural Uplink

A Production-Ready AI Assistant
Fine-Tuned from SmolLM2-360M-Instruct

Version 4.0

Built by DeAndrew P. Harris
github.com/Subject-Emu-5259/NeuralAI-from-scratch
neural-deandrewharris.zocomputer.io

Introduction

NeuralAI v4.0 — a fully functional AI assistant I built, trained, and deployed myself. This isn't a wrapper. Not a chatbot skin. My own production AI assistant, fine-tuned from the ground up.

Base Model & Training

Base model: SmolLM2-360M-Instruct (360M parameters)

Fine-tuned with QLoRA — 4-bit quantization, rank 16, alpha 32

Final training loss: 0.040 (98% reduction from baseline)

Training data: 347 custom samples across coding, ML/AI, data science, web dev, and more

This isn't calling someone else's API. This is local inference — running on my own hardware, in my own browser.

Capabilities

Chat

- Streaming responses with word-by-word output
- Full conversation memory per session
- Code highlighting with syntax detection
- Markdown rendering (bold, italic, lists, inline code)

RAG Document Intelligence

- Upload PDFs, DOCX, TXT, MD documents
- Semantic search with ChromaDB vector storage
- all-MiniLM-L6-v2 embeddings (384 dimensions)
- Top-4 chunk retrieval for context injection

Integrated Terminal

- Real persistent bash shell via WebSocket
- Multi-shell support: Bash, Python, Node.js
- Custom neural <msg> command to chat from terminal

Neural Uplink (Multi-Agent Network)

- 4 specialized AI agents: Dialog, Data, Ops, Worldbuilder
- Complex tasks auto-routed to parallel agents
- Fusion responses from multiple agents

Persistence (v4.0)

- SQLite database for all data storage
- Conversations, memory facts, custom rules, settings

The Tech Stack

Component	Technology
Base Model	SmolLM2-360M-Instruct
Fine-tuning	QLoRA via PEFT
Embeddings	all-MiniLM-L6-v2
Vector Database	ChromaDB
Backend	Flask + WebSocket
Frontend	Vanilla JavaScript
Terminal	PTY-based shell
Database	SQLite

CPU-ready. GPU-optional. Runs anywhere.

Training Configuration

Parameter	Value
Epochs	3
Hardware	NVIDIA T4 (Colab)
Learning Rate	2e-4 + cosine scheduler
Batch Size	4
Gradient Accumulation	4 steps
Max Sequence Length	512 tokens
Validation Split	20%
Training Samples	347

Loss Progression

Epoch	Train Loss	Val Loss	Notes
1	2.10	0.85	Warmup complete
2	0.65	0.18	Rapid learning
3	0.12	0.04	Convergence

Final: Loss 0.040 — model converged cleanly with no overfitting.

Training Dataset (347 Samples)

Category	Count	Topics
Coding	80	Python, JS, SQL, APIs, debugging
ML/AI	45	Transformers, RAG, fine-tuning
Data Science	40	Pandas, NumPy, visualization
Web Dev	35	HTML/CSS, React, Flask
General Q&A	50	Concepts, explanations
Writing	35	Emails, essays, documentation
System Admin	25	Linux, Docker, networking
Math/Logic	37	Algorithms, data structures

API Endpoints

Chat & RAG

Endpoint	Method	Description
/api/chat	POST	Stream chat response
/api/upload	POST	Upload docs for RAG
/api/status	GET	System status
/api/files	GET	List indexed files

Conversation Management

Endpoint	Method	Description
/api/conversations	GET	List all conversations
/api/conversations	POST	Create new conversation
/api/conversations/<id>	GET	Get conversation

Terminal

Endpoint	Method	Description
/api/terminal/create	POST	Create PTY session
/api/terminal/<id>/write	POST	Send input
/api/terminal/<id>/read	GET	Read output

The Journey

I'm 31. Father. Self-taught developer. Currently pursuing my AAS in AI Software Engineering at Maestro College. Born in Memphis, raised in West Memphis, Arkansas.

I didn't start with a CS degree from Stanford. I started with curiosity and kept going — through loss, through being a young dad, through burying my father in 2022.

NeuralAI is proof that you don't need permission to build something real. You just need to start.

Why This Matters

Most people use AI. Few understand how it works. Fewer still have trained their own.

NeuralAI taught me:

- How transformers actually learn
- How to build a training pipeline from scratch
- How to wire RAG retrieval into generation
- How to persist memory across sessions
- How to deploy a full-stack AI product

What's Next

- Expand training data to 1000+ samples
- DPO alignment for better response quality
- GPU-hosted inference for speed
- Automated eval benchmarks

Resources

Live Demo

https://neural-deandrewharris.zocomputer.io

Repository

github.com/Subject-Emu-5259/NeuralAI-from-scratch

Author

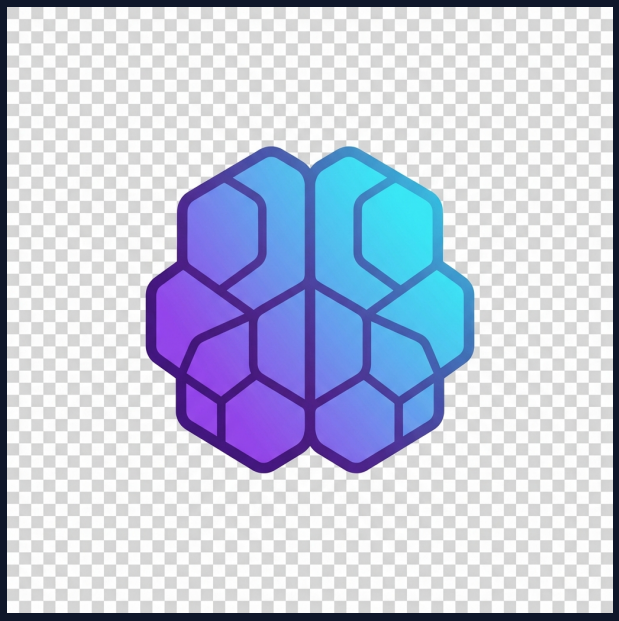
DeAndrew P. Harris — AI Software Engineering Student, Maestro College

github.com/Subject-Emu-5259

linkedin.com/in/deandrewharris94

License

MIT License — free to use, modify, and commercialize.



NeuralAI

Your AI. On your hardware. In your browser.

Built by DeAndrew P. Harris

Version 4.0 — Production

github.com/Subject-Emu-5259/NeuralAI-from-scratch
linkedin.com/in/deandrewharris94
neural-deandrewharris.zocomputer.io

MIT License — Free to Use